

RLSP & Emergence of Thinking: Experiment Pipeline Summary

Αυτό το notebook υλοποιεί ένα υτοματοποιημένο pipeline 6 **σειριακών πειραμάτων**. Στόχος είναι η αναπαραγωγή της Chain-of-Thought μέσω Reinforcement Learning (PPO) σε ένα μοντέλο 1.5B παραμέτρων, σε περιβάλλον μίας μόνο κάρτας γραφικών (1x L4 24GB).

1. Datasets & Γνωστικοί Τομείς

Το μοντέλο αξιολογείται σε τρεις διακριτούς τομείς για να αποδειχθεί η γενίκευση της ικανότητας συλλογισμού:

- **Μαθηματικά (Math500)**: Προβλήματα που απαιτούν αναλυτική αλγεβρική επίλυση. Η απάντηση εξάγεται από `format \boxed{}`.
- **Φυσική (SciBench / xw27)**: Πανεπιστημιακά προβλήματα φυσικής (θερμοδυναμική, κβαντική, κ.α.). Ελέγχεται η ακρίβεια του αριθμητικού αποτελέσματος με ανοχή απόκλισης (tolerance) 5%.
- **Λογική (ReClor)**: Προβλήματα κατανόησης και λογικής αφαίρεσης/επαγωγής από κείμενα (debates). Το μοντέλο καλείται να επιλέξει τη σωστή απάντηση (0, 1, 2, ή 3).

2. Curriculum vs. Baseline (Reward Functions)

Για κάθε Dataset τρέχουν δύο παραλλαγές (Συνολικά 6 Runs), συγκρίνοντας το Baseline του paper με την προτεινόμενη μέθοδο:

A. RLSP Baseline (Static Penalty)

- **Λογική**: Το παραδοσιακό PPO ενισχυμένο με μια σταθερή ποινή μήκους (Length Penalty).
- **Μαθηματικός Τύπος**: $Reward = Outcome - \frac{20}{Tokens_{reasoning}}$
(Όπου $Outcome = 1.0$ αν η απάντηση είναι σωστή, αλλιώς 0.0).
- **Υλοποίηση**: Χρησιμοποιούνται Multiprocessing Flask Servers (βασισμένοι στον `orm_server_efficient.py` του paper) με σταθερό `--length_penalty 20.0`.

B. Model Paper (Dynamic Curriculum)

- **Λογική**: Το μοντέλο μαθαίνει να "σκέφτεται" σταδιακά. Αρχικά επιβραβεύεται η "εξερεύνηση" (η παραγωγή μεγάλων κειμένων), και σταδιακά η επιβράβευση μετατοπίζεται αποκλειστικά στην ορθότητα της απάντησης.
- **Μαθηματικός Τύπος**: $Reward = \alpha \cdot Outcome + (1 - \alpha) \cdot \left(-\frac{20}{Tokens+1}\right)$
- **To Curriculum (α)**: Το βάρος α ξεκινάει από 0.2 και αυξάνεται γραμμικά (Linear Interpolation) μέχρι το 1.0. Η κορύφωση στο 1.0 συμβαίνει ακριβώς στο 800ό ερώτημα (τελευταίο βήμα της εκπαίδευσης).

⇒ ΣΥΓΚΕΚΡΙΜΕΝΑ

Χρησιμοποιούμε την εξής λογική:

Έστω q το τρέχον βήμα (αριθμός ερωτημάτων) και Q_{max} το μέγιστο όριο βημάτων της εκπαίδευσης.

1. Ορίζουμε το ποσοστό προόδου t ως: $t = \min\left(\frac{q}{Q_{max}}, 1\right)$
2. Το δυναμικό βάρος $\alpha(q)$ υπολογίζεται μέσω linear interpolation (⇒ aka απλή μέθοδος των 3ων) μεταξύ της αρχικής τιμής α_{start} και της τελικής α_{end} :

$$\alpha(q) = \alpha_{start} + t \cdot (\alpha_{end} - \alpha_{start})$$

(ΠΗΡΑ: $\alpha_{start} = 0.2$ και $\alpha_{end} = 1.0$)

3. Παραμετροποίηση & Hardware Optimization

Το pipeline έχει βελτιστοποιηθεί χειρουργικά για να χωρέσει τα 4 μοντέλα του PPO (Actor, Critic, Reference, Reward) στα 24GB VRAM της NVIDIA L4 ([Lightning.ai](#)):

- **Ray & GPU Hack:** Γίνεται χρήση 2 "Virtual GPUs" μέσω Ray (`--num-gpus 2`) πάνω στην 1 φυσική κάρτα (`CUDA_VISIBLE_DEVICES=0`).
- **VRAM Savings:** Απενεργοποίηση του vLLM (`--vllm_num_engines 0`) και χρήση bf16 (`--bf16`).
- **Batching:** * `rollout_batch_size: 32`
 - `train_batch_size: 16`
 - `micro_rollout_batch_size: 4`
 - `micro_train_batch_size: 4`
- **Μοντέλο & Εκπαίδευση:** Qwen/Qwen2.5-1.5B-Instruct με εφαρμογή **LoRA** (`rank=64`, `alpha=128`) σε όλα τα projection layers (q, k, v, o).
- **Διάρκεια Εκπαίδευσης (Ανά Πείραμα):** `max_epochs: 1`, `num_episodes: 25`.
(Σύνολο: $25 \times 32 = 800$ queries).

4. Χρόνοι Εκτέλεσης & Διαχείριση ⇒ Θα δολυμε αυτα ειναι rough estimates

- **Χρόνος ανά Πείραμα:** ΠΛΗΡΩΣ ΑΙΣΙΟΔΟΞΑ ~1.5 με 2 ώρες. **To generation των max 1024 tokens χωρίς vLLM είναι το πιο υπολογιστικά βαρύ στάδιο ==> ΜΗΠΩΣ ΝΑ Ο ΑΛΛΑΞΟΥΜΕ.**
- Αν και εγω πιστευω ΡΕΑΛΙΣΤΙΚΑ καθε πειρμα θα παρει ~14h
- Προφανως σε ενα sessions δεν μπουρουν να τρεξουν ΟΛΑ τα πειραματα.
- **Διαχείριση Πόρων:** Ανάμεσα σε κάθε ένα από τα 6 πειράματα, το Python script εκτελεί αυτόματα `teardown` (`pkill, ray stop --force`) για να αδειάσει πλήρως η VRAM και να απελευθερωθούν τα HTTP Ports (8000, 8001, 8002) των Flask ORM Servers.